1 Installation Manual

1.1 Prerequisites

- `git`
- Docker Desktop (macOS / Windows) or Docker Engine + Docker Compose (Linux)
- macOS / Linux only: `bash` and `curl`
- Make sure these local ports are available. Or you change them in the root `.env` file: 8080, 3000, 5050, 15432, 1025, 8025

You do not need to install PostgreSQL, Prisma, or Node.js locally. We use Docker for the setup.

1.2 Quick Start

1. **Clone the repository**

```
git clone https://github.com/unsw-cse-comp99-3900/capstone-project-26t1-9900-
    f09c-cake.git
cd capstone-project-26t1-9900-f09c-cake
```

2. **Create the environment file**

   Create a root `.env` file from the provided example:

   macOS / Linux:

```
cp .env.example .env
```

   Windows (PowerShell):

```
Copy-Item .env.example .env
```

The default values in `.env.example` are sufficient for local marking and development. Do not commit the generated `.env` file to GitHub.

1.3 Start the Application

The recommended startup scripts build the Docker containers, start the services, apply the Prisma database schema, seed demo data, and wait for health checks.

macOS / Linux:

```
bash start-local.sh
```

Windows (PowerShell):

```
.\start-local.ps1
```

First run takes **3–5 minutes** due to Docker image downloads. Subsequent starts take under 30 seconds.

If you need to run Docker Compose manually instead of using the scripts:

```
docker compose --env-file .env -p tutorconnect up --build -d
docker compose --env-file .env -p tutorconnect exec -T backend pnpm exec prisma db
    push --accept-data-loss
docker compose --env-file .env -p tutorconnect exec -T backend pnpm run db:seed
```

1.4 Access the Application

Once the script finishes, open your browser:

Service	URL
Application	`http://localhost:8080`
API Health Check	`http://localhost:8080/api/v1/health`
Email Inbox (dev)	`http://localhost:8025`

If one of those ports is already in use, update the matching port variable in the root `.env` file and rerun the startup script.

1.5 Demo Accounts

The database is pre-seeded with the following accounts for testing:

Role	Email	Password
Admin	`admin@tutorconnect.com`	`Admin@123456`
Learner	`learner@demo.com`	`Learner@123`
Tutor (verified)	`tutor@demo.com`	`Tutor@123`
Tutor (pending review)	`pending.tutor@demo.com`	`Tutor@123`
Learner (LA timezone)	`tz.learner.la@demo.com`	`Demo@123456`
Tutor (Sydney timezone)	`tz.tutor.sydney@demo.com`	`Demo@123456`

Email verification emails are captured by the local mail server at `http://localhost:8025` (no real emails are sent.)

1.6 Environment Variables and Secrets

The application expects one environment file at the project root:

```
.env
```

For local marking, create it by copying `.env.example`. The project repository includes `.env.example`, but the real `.env` file is intentionally ignored by Git.

Variable	Required for local run	Secret	Purpose / Example
FRONTEND_URL	Yes	No	Frontend URL used in links and CORS, default http://localhost:8080
CORS_ORIGIN	Yes	No	Comma-separated allowed browser origins, default http://localhost:3000,http://localhost:8080
EMAIL_VERIFICATION_REQUIRED	Yes	No	Enables email verification flow, default true

Variable	Required for local run	Secret	Purpose / Example
JWT_SECRET	Yes	Yes	JWT access-token signing secret. Local default is dev-only; replace in production.
JWT_REFRESH_SECRET	Yes	Yes	JWT refresh-token signing secret. Local default is dev-only; replace in production.
EMAIL_FROM	Yes	No	Sender shown in verification emails
SMTP_HOST	Local default provided	No	SMTP host. Leave blank or use Mailpit defaults locally.
SMTP_PORT	Yes	No	SMTP port, default 1025 for Mailpit
SMTP_SECURE	Yes	No	SMTP TLS flag, default false locally
SMTP_USER	No	Yes	SMTP username, optional locally
SMTP_PASS	No	Yes	SMTP password, optional locally
NEXT_PUBLIC_API_URL	Yes	No	Frontend API path, default /api/v1
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	No	No	Stripe publishable key. Optional for local payment testing.
STRIPE_SECRET_KEY	No	Yes	Stripe secret key. Optional locally; required only for live Stripe payment testing.
STRIPE_WEBHOOK_SECRET	No	Yes	Stripe webhook signing secret. Required if testing real Stripe webhooks.
POSTGRES_HOST_PORT	Yes	No	Host port for PostgreSQL, default 15432
BACKEND_HOST_PORT	Yes	No	Host port for backend API, default 5050
FRONTEND_HOST_PORT	Yes	No	Host port for the frontend container, default 3000
NGINX_HOST_PORT	Yes	No	Host port for the unified application URL, default 8080
MAILPIT_SMTP_HOST_PORT	Yes	No	Host port for Mailpit SMTP, default 1025
MAILPIT_UI_HOST_PORT	Yes	No	Host port for Mailpit web inbox, default 8025

Figure 1: Variable2

No production secrets are stored in the GitHub repository. The local defaults in **.env.example** are enough to run the application and use the demo accounts. If Stripe is assessed, put the Stripe test keys in the local root .env file only. Do not put them in GitHub.

Example Local `.env`

```
FRONTEND_URL=http://localhost:8080
CORS_ORIGIN=http://localhost:3000,http://localhost:8080
EMAIL_VERIFICATION_REQUIRED=true
JWT_SECRET=local-dev-jwt-secret-change-me
JWT_REFRESH_SECRET=local-dev-jwt-refresh-secret-change-me
EMAIL_FROM=TutorConnect <no-reply@example.com>
SMTP_HOST=
SMTP_PORT=1025
SMTP_SECURE=false
SMTP_USER=
SMTP_PASS=
NEXT_PUBLIC_API_URL=/api/v1
```

```
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
POSTGRES_HOST_PORT=15432
BACKEND_HOST_PORT=5050
FRONTEND_HOST_PORT=3000
NGINX_HOST_PORT=8080
MAILPIT_SMTP_HOST_PORT=1025
MAILPIT_UI_HOST_PORT=8025
```

Stripe Configuration: If you want to use Stripe, you need to update these three values in the `.env` file:

```
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=
pk_test_51TKeQzPWbP1IPgAagdxpYTL9I89PB76u87Fp154QMaS7Hg4FaDt2nxS0XXHSxmlJZLuUcZdSr
G26wClZDoLnrHBl002YzgEAJv

STRIPE_SECRET_KEY=
sk_test_51TKeQzPWbP1IPgAaeC43ZKzpuXiTCrUaZrW2b4O0V9rMl9DJS74H8d5F1K9wvI9vx20IaSgQl
B57HlyMtWRkaiC500I5HfUXEQ

STRIPE_WEBHOOK_SECRET=
```

Note:

For production-style deployment, replace `JWT_SECRET`, `JWT_REFRESH_SECRET`, database credentials, SMTP credentials, and Stripe secrets with unique secure values.